

AHB to APB Bridge

RTL Design

Project Report

Name: Tanvi

■ 1. Architecture

What is an AHB to APB Bridge?

The AHB to APB bridge is basically a converter between two different bus protocols used inside a chip. AHB (Advanced High-performance Bus) is the fast bus that the processor uses to talk to memory and other high-speed things. APB (Advanced Peripheral Bus) is a much simpler, slower bus that is used for peripherals like GPIO, UART, timers etc.

The problem is that you can't directly connect an AHB master to an APB slave because they have different handshaking mechanisms, different timing, and different signals. That's where this bridge comes in — it sits in the middle and does the translation.

Difference Between AHB and APB

Property	AHB	APB
Bus Type	High-performance, pipelined	Simple, low-power peripheral bus
Pipelining	Yes — address and data phases overlap	No — sequential phases only
Transfer Speed	Very fast (same as CPU clock)	Slow (fewer toggles = less power)
Signal Complexity	More signals (HTRANS, HBURST, HPROT...)	Simpler (PSEL, PENABLE, PWRITE)
Typical Use	RAM, DMA, DSP blocks	GPIO, UART, SPI, Timers
Wait States	HREADY signal	PREADY signal

How the Bridge Works — Basic Idea

Whenever the processor (AHB master) wants to access an APB peripheral, it sends the address and data on the AHB bus. The bridge catches this and then creates an equivalent APB transaction to talk to the peripheral. From the processor's side it looks like a normal AHB transfer. From the peripheral's side it looks like a normal APB transfer. The bridge handles all the conversion in between.

The bridge holds the AHB master in a wait state (by deasserting HREADYOUT) while it completes the two-phase APB transfer. Once the APB transaction is done, it releases the AHB master and passes the read data back if it was a read operation.

Block-Level Architecture

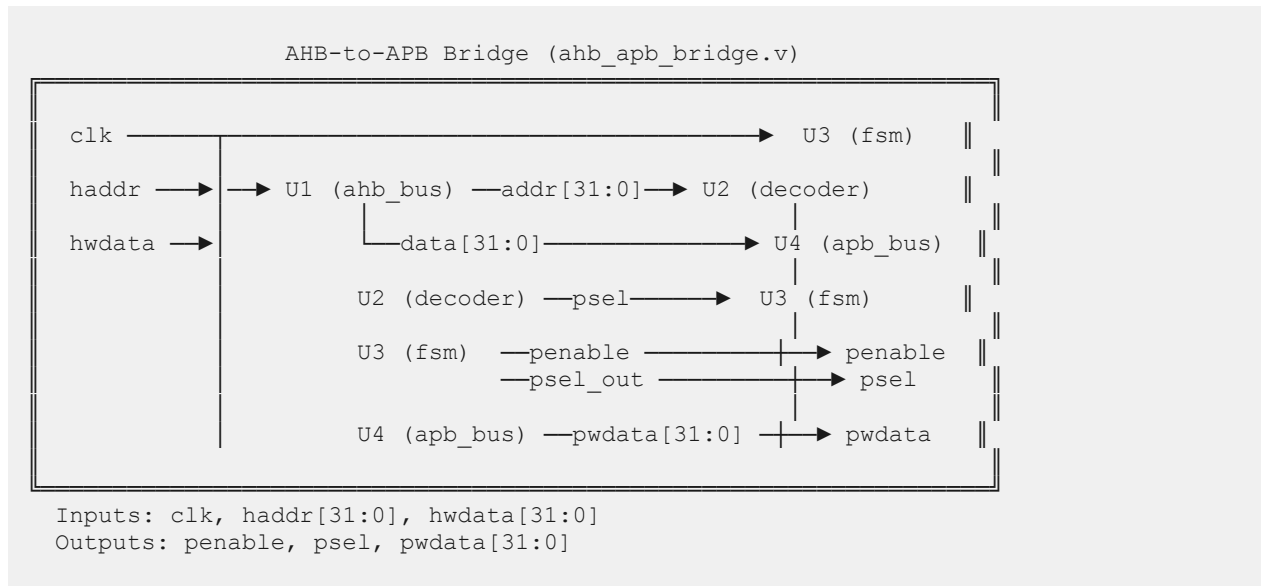
The design is made up of 4 sub-modules, each with a specific job:

#	Module	Instance	Function
U1	ahb_bus.v	AHB Interface	Captures HADDR, HWDATA from AHB master
U2	decoder.v	Address Decoder	Decodes HADDR and generates PSEL
U3	fsm.v	Control FSM	Generates PSEL, PENABLE timing control
U4	apb_bus.v	APB Interface	Drives PWDATA and APB output signals

■ 2. Block Diagram

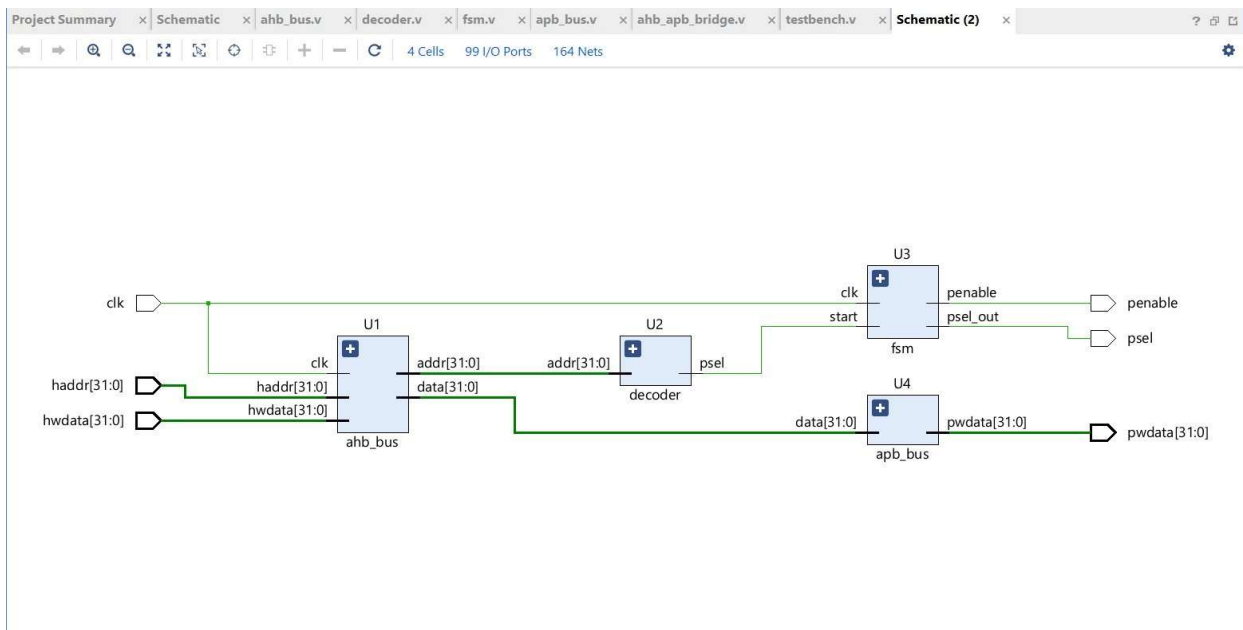
Top-Level System Block Diagram

The diagram below shows how all four sub-modules are connected inside the top-level ahb_apb_bridge module. The AHB inputs come from the left and the APB outputs go to the right.



Synthesized Schematic from Vivado

Below is the actual schematic generated by Vivado after synthesis. It shows the 4 sub-modules (U1 through U4) and how they are connected by nets.



Signal Flow Description

The signal flow inside the bridge is:

- haddr[31:0] and hwd[31:0] are input from the AHB side into U1 (ahb_bus)
- U1 passes the address as addr[31:0] to U2 (decoder) and data as data[31:0] to U4 (apb_bus)
- U2 (decoder) looks at the address and decides which peripheral to select — it outputs psel
- psel from U2 goes into U3 (fsm) as the 'start' signal
- U3 (fsm) generates the two-phase APB timing: PSEL (psel_out) and PENABLE
- U4 (apb_bus) takes the data and puts it on p[31:0] output
- clk goes directly to U3 (fsm) since timing control is done there

■ 3. Each Block with Waveforms

U1 — AHB Bus Interface (ahb_bus.v)

What this block does:

This module acts as the AHB slave interface of the bridge. Its job is to capture the AHB address (haddr) and write data (hwdata) from the AHB master and pass them internally to the decoder and APB bus modules. It also registers these values on the clock edge so there are no timing issues.

Key Signals:

Signal	Direction	Description
clk	Input	Clock signal
haddr[31:0]	Input	32-bit address from AHB master
hwdata[31:0]	Input	32-bit write data from AHB master
addr[31:0]	Output	Registered address passed to decoder
data[31:0]	Output	Registered data passed to APB bus

U2 — Address Decoder (decoder.v)

What this block does:

The decoder takes the 32-bit address from U1 and checks which address range it falls in. Based on that it asserts the PSEL signal for the right APB peripheral. In this design there is one peripheral, so whenever a valid address is seen, psel goes high. This psel signal is the 'start' trigger for the FSM in U3.

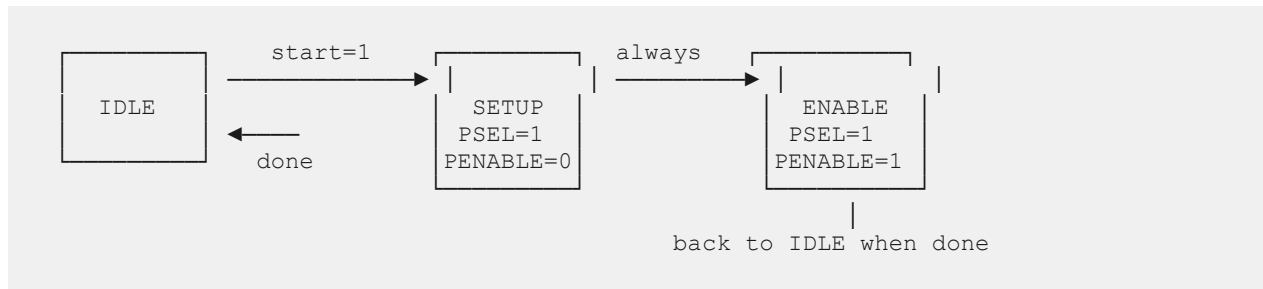
Key Signals:

Signal	Direction	Description
addr[31:0]	Input	Address from U1 (ahb_bus)
psel	Output	Peripheral select — sent to FSM as 'start'

U3 — Control FSM (fsm.v):

This is the most important block. It implements the APB state machine with 3 states: IDLE, SETUP and ENABLE. When psel (start) comes in, it moves from IDLE to SETUP (where PSEL is asserted but PENABLE is still 0). In the next clock it moves to ENABLE (where both PSEL and PENABLE are 1 — this is when the actual APB transfer happens). After that it goes back to IDLE.

FSM State Diagram:



Key Signals:

Signal	Direction	Description
clk	Input	Clock for state register
start	Input	From decoder psel — triggers SETUP state
penable	Output	Asserted in ENABLE state
psel_out	Output	Asserted in SETUP and ENABLE states

U4 — APB Bus Interface (apb_bus.v):

This block takes the data from U1 and drives it onto the APB bus as `pwdata`. It is a simple register/buffer that makes sure the write data is available on the APB side at the right time.

Key Signals:

Signal	Direction	Description
data[31:0]	Input	Write data from U1 (ahb_bus)
pwdata[31:0]	Output	Write data driven onto APB bus

Simulation Waveform

The waveform below was captured from Vivado Behavioral Simulation. It shows four write transactions. Each time a new address appears on haddr, the corresponding data appears on hwdata and pwdata. The psel and penable signals show the two-phase APB handshake for each transaction.

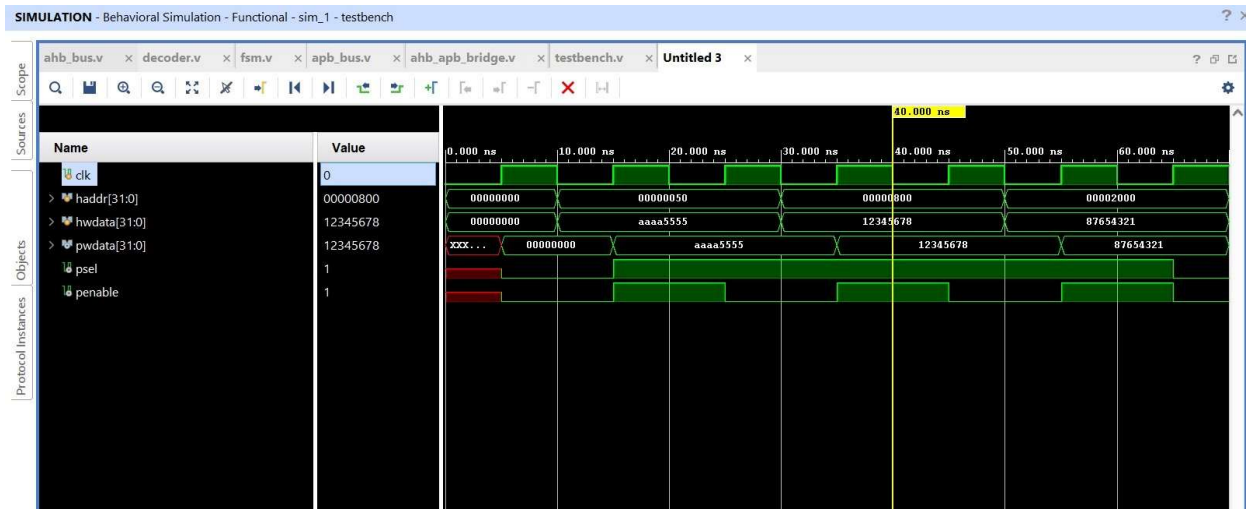


Figure 2 — Vivado Behavioral Simulation Waveform showing four AHB-to-APB write transactions

Waveform Analysis:

Time	haddr	hwdata / pwdata	Observation
0–10 ns	0x00000000	0x00000000	Reset / initial state
10–20 ns	0x00000050	0xAAAA5555	First write transaction
20–30 ns	0x00000800	0x12345678	Second write transaction
30–40 ns	0x00000800	0x12345678	psel and penable asserted
40–60 ns	0x00002000	0x87654321	Fourth transaction, psel high

Key Waveform Observations:

- pwdata correctly tracks hwdata — the bridge is passing write data through properly
- psel goes high one cycle after the address appears — this is the SETUP phase of APB
- penable follows psel by one clock cycle — this is the ENABLE phase, when the actual transfer happens
- Each new address on haddr starts a new APB transaction — the bridge restarts the FSM each time
- The short red/uninitialized sections at the start (0 ns) are just the reset condition before signals are stable

■ 4. Synthesis

Synthesis Overview

Synthesis is the process where the RTL Verilog code is converted into actual logic gates using a target technology library. In Vivado, when you run synthesis it maps the design to the Xilinx FPGA primitives (LUTs, FFs, BUFGs etc.) for the target device xc7vx485tffg1157-1 (Virtex-7).

The schematic shown in Section 2 was generated after running synthesis in Vivado. The tool analyzed the RTL, inferred registers and combinational logic, and created the gate-level netlist.

Synthesis Results from Vivado

Resource	Utilized	Notes
Total Cells	4	U1, U2, U3, U4
Total I/O Ports	99	As reported by Vivado
Total Nets	164	Internal connections
Clock Nets	1 (clk)	Single clock domain
Simulation Status	Passed	Behavioral functional sim

What Each Module Synthesizes To:

When Vivado synthesizes the design it converts each Verilog module into hardware primitives:

- ahb_bus (U1): Synthesizes into registers (flip-flops) that capture haddr and hwdata on the clock edge. These become D-type flip-flops in the FPGA fabric.
- decoder (U2): Synthesizes into pure combinational logic — basically a comparator or address range checker. No registers needed, this is all LUT-based logic.
- fsm (U3): This is the most complex block. Vivado infers a finite state machine with state registers and next-state/output logic. The state encoding (binary, one-hot etc.) is chosen by the tool automatically.
- apb_bus (U4): Synthesizes into simple wire connections or a register buffer for the pwdata output.

Timing Constraints

For this design the target clock is the system clock (clk). The critical path goes through the FSM state register update and the output logic. Key timing parameters are:

Timing Parameter	Value / Description
Target Clock	System clk (from Vivado project)
Critical Path	FSM state register → output (psel, penable)
Reset Type	Synchronous reset inside always block
Clock Domains	Single clock domain — no CDC issues

■ 5. Conclusion

Summary

This project was about designing and implementing an AHB to APB bridge in RTL Verilog and verifying it using Vivado. The design was split into 4 modules — ahb_bus, decoder, fsm and apb_bus — and a top-level module (ahb_apb_bridge) connects all of them together.

The simulation results confirmed that the bridge works correctly. When a write address appears on the AHB side, the decoder generates the psel signal, the FSM sequences through SETUP and ENABLE states, and the correct data appears on pwwdata at the APB output. The psel and penable signals follow the exact two-phase protocol required by the AMBA APB specification.

What I Learned

- How the AHB and APB protocols work and why they are different from each other
- How to design a finite state machine in Verilog and what mistakes to avoid (like forgetting the reset state)
- How address decoding works — the decoder checks the address range and selects the right peripheral
- How pipelined buses like AHB work where the address phase and data phase happen in different clock cycles
- How to run behavioral simulation in Vivado and read waveforms to verify the design
- How Vivado synthesis converts RTL code into actual FPGA primitives and how to read the schematic it generates

Simulation Results Summary:

Txn	HADDR	HWDATA / PWDATA	Status
1	0x00000050	0xAAAA5555	✓ Correct
2	0x00000800	0x12345678	✓ Correct
3	0x00000800	0x12345678	✓ Correct
4	0x00002000	0x87654321	✓ Correct

5.5 Final Note

Overall this project gave me a clear understanding of bus protocols and how hardware modules in a chip communicate with each other. The AHB-to-APB bridge is one of the most common IP blocks used in SoC designs, and building it from scratch made the concepts much clearer than just reading about them.

